Module 8 - SQL Access Control (DCL)

& Database Administration

Learning Outcomes

By the end of this module, you will be able to:

- Explain the role and key responsibilities of a **Database Administrator (DBA)**
- Describe **users, roles, privileges, and profiles**
- Use **GRANT** and **REVOKE** to control access to database objects
- Apply **views** and **roles** to enforce security
- Integrate access-control principles into your **project database**

Why Access Control?

Imagine your Lab 5 database is shared among:

- Students updating their own records
- Instructors reviewing all submissions
- Admins managing terms and courses

➡ Should they all have the same access?

Access control ensures security, privacy, and integrity.

The DBA Role

Database Administrator (DBA)

Responsible for:

- Creating and maintaining the database environment
- Managing users and roles
- Controlling storage and performance parameters
- Enforcing security and backup policies

🟦 The DBA is the guardian of the data ecosystem.

The Database Ecosystem

Role	Main Task	Example
Database Designer	Builds conceptual & logical models	EERD, schema
Developer	Writes application code	Web app, SQL scripts
DBA	Maintains DB operation & security	User management
End User	Uses applications	Runs queries & reports

Common DBA Tasks

- Start and stop database services
- Create users and roles
- Grant and revoke privileges
- Manage storage (tablespaces, files)
- Monitor performance and logs
- Configure backups and recovery

Database Services

Every DBMS runs background services:

- **Listener service** – accepts connections
- **Instance** – running copy of DB in memory
- **Processes** – handle user queries and transactions

🟦 The DBA ensures these start automatically and run smoothly.

Managing Users

- A **user** is a unique account that can log into the DB.
- Each user is identified by a login ID and has privileges assigned by the DBA.
- Users can own database objects and grant permissions to others.

```
CREATE USER alex PASSWORD 'abcd1234';
```

Managing Roles

- A **role** is a named set of privileges.
- Roles simplify user management.
- Common roles:
  - `CONNECT` – basic login and object creation
  - `RESOURCE` – ability to create procedures, triggers
  - `DBA` – full administrative control

```
CREATE ROLE instructor;
GRANT CONNECT TO instructor;
GRANT SELECT, UPDATE ON enrollment TO instructor;
```

Users vs. Roles vs. Privileges

Concept	Definition	Example
User	Account that logs in	`maryam`
Role	Group of privileges	`student_role`
Privilege	Specific permission	`SELECT ON lab_event`

Privileges Overview

Privileges define **what actions** a user can perform:

Action	SQL Privilege	Example
Read data	`SELECT`	`SELECT ON student`
Add data	`INSERT`	`INSERT ON enrollment`
Change data	`UPDATE`	`UPDATE ON progress`
Delete data	`DELETE`	`DELETE ON progress`

Ownership and Delegation

- Table creator = **owner** and holds all privileges.
- The owner may delegate privileges to others.
- Privileges can be **passed on** using the `WITH GRANT OPTION` clause.

Example Scenario

You own table `course_offering`.

```
GRANT SELECT, INSERT ON course_offering TO instructor_role
GRANT UPDATE ON course_offering TO admin_role WITH GRANT O
```

🟦 The `admin_role` can now grant UPDATE privileges to others.

Revoking Privileges

```sql
REVOKE UPDATE ON course_offering FROM admin_role CASCADE;
```

- `CASCADE` : also revokes privileges that were granted indirectly.
- `RESTRICT` : default – fails if other privileges depend on it.

Privileges Example

Question: What privileges are required to run this statement?

```
UPDATE APPLY
   SET DECISION = 'Y'
 WHERE S_ID IN (SELECT S_ID FROM STUDENT WHERE GPA > 3.9);
```

✅ Required privileges:

- `UPDATE(decision)` on `APPLY`
- `SELECT(S_ID, GPA)` on `STUDENT`

Common Privileges

Privilege	Description	Applies To
`SELECT`	Read data from a table or view	Tables, Views
`INSERT`	Add new rows	Tables
`UPDATE`	Modify existing data	Tables, Columns
`DELETE`	Remove rows	Tables
`REFERENCES`	Create foreign keys referencing a table	Tables
`EXECUTE`	Run a stored procedure or function	Functions
`USAGE`	Use sequences, schemas, or data types	Schemas, Sequences
`ALL`	Grant all applicable privileges	All Objects

Full GRANT syntax

```
GRANT privilege [, privilege ...]
ON { table_name | view_name | sequence_name | function_nam
TO { role_name | user_name | PUBLIC }
[WITH GRANT OPTION];
```

Example:

```
GRANT SELECT, INSERT, UPDATE, DELETE, REFERENCES
ON student
TO instructor_role WITH GRANT OPTION;
```

Defining a Privilege

Every privilege can be scoped to a specific object and optionally columns:

```
GRANT UPDATE(column_name [, column_name ...])
ON table_name
TO user_or_role;
```

Examples:

```
-- Grant UPDATE only on GPA column
GRANT UPDATE(gpa) ON student TO instructor_role;

-- Grant INSERT on entire table
GRANT INSERT ON enrollment TO student_role;
```

🟦 Column-level control allows precise permissions while protecting other data.

Using Views for Security

Views can restrict columns or rows accessible to users.

```
CREATE VIEW bcit_students AS
SELECT s_id, name, gpa
FROM student
WHERE school_name = 'BCIT';

GRANT SELECT ON bcit_students TO staff;
```

🟦 Staff can now see only BCIT students.

Views as Access Layers

Base tables → Secure Views → End Users

- Simplifies queries
- Restricts sensitive data exposure
- Supports the principle of least privilege

DCL Syntax Summary

```sql
GRANT privilege_list ON object
TO user_or_role [WITH GRANT OPTION];

REVOKE privilege_list ON object
FROM user_or_role [CASCADE | RESTRICT];
```

Common privileges: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `REFERENCES`, `ALL`.

Discussion Prompt

💬 Why is WITH GRANT OPTION risky?

Discussion Prompt

💬 Why is WITH GRANT OPTION risky?

➡ It can cause **privilege sprawl** if users re-grant permissions carelessly.

DB Authorization Summary

- **GRANT** privileges to users or roles.
- **REVOKE** privileges to remove access.
- **Views** can be used as access filters.
- **Roles** simplify privilege management.

Integrating DCL into Projects

Define clear roles within your project schema:

Role	Privileges
admin	All DDL/DML; manage users; grant privileges
instructor	View and update enrollment data
student	Insert/update their own enrollment records only

🟦 Encourage students to implement and test roles.

Applying Access Control

```sql
CREATE ROLE student_role;
GRANT SELECT ON course TO student_role;
GRANT INSERT, UPDATE ON enrollment TO student_role;

CREATE ROLE instructor_role;
GRANT SELECT, UPDATE ON enrollment TO instructor_role;
GRANT SELECT ON student TO instructor_role;
```

Testing Access

Switch between roles to validate privileges:

```
SET ROLE instructor_role;
SELECT * FROM enrollment;   -- ✅ Works
DELETE FROM enrollment;     -- ❌ Permission denied
RESET ROLE;
```

Use `SET SESSION AUTHORIZATION` to simulate individual users.

Roles & Privileges Best Practices

- Grant privileges to **roles**, not users.
- Follow the **principle of least privilege**.
- Revoke unused privileges regularly.
- Document all roles and grants.

Knowledge Check

1. What is the difference between a role and a user?
2. When would you use `WITH GRANT OPTION`?
3. How do views enhance database security?
4. Why should privileges be granted to roles instead of users?

Reflection

💬 Which tables in your schema should be admin-only?
💬 What views could you create to support instructors or students?

Summary

- **DBA** manages users, roles, and security policies.
- **DCL** commands control object access.
- **Views** and **roles** provide fine-grained control.
- Apply these principles to your project schema.

Reading & Resources

Source	Chapter / Section
Elmasri & Navathe	Database Security & Authorization
Silberschatz et al.	Ch. 16 – Security & Access Control
Beaulieu, Learning SQL	Ch. 13 – Security and Access Control
OER Database Design	Ch. 8 – Security, Privileges, and Roles